

TéléPro CI

Déploiement SaaS — Plan de migration

Architecture multi-tenant · PostgreSQL Schema-per-tenant · Avril 2026

En bref

Cette note détaille les migrations PostgreSQL et les adaptations API nécessaires pour transformer TéléPro CI en application SaaS multi-tenant. L'approche retenue est le schema-per-tenant : chaque client possède son propre schéma PostgreSQL isolé. Le code métier existant reste inchangé — seul un middleware de routage est ajouté.

Généré le 20 avril 2026

1. Comparaison des approches multi-tenant

Trois architectures sont possibles pour un SaaS PostgreSQL. Les critères clés pour TéléPro CI (marché Afrique de l'Ouest, PME, v

Critère	tenant_id partout	& Schema/tenant	Base/tenant
Isolation des données	Faible — risque oubli	& Forte — garantie DB	Maximale
Modifications du code	+ 30 colonnes + RLS	& Middleware seul	Aucune
Migrations par client	Une fois globale	Runner par tenant	Séquence complète
Backup par client	Difficile	& pg_dump -n schema	pg_dump complet
Nb tenants supporté	Illimité	& < 5 000	< 200
Risque de fuite de données	Élevé (oubli WHERE)	& Quasi nul	Nul
Coût infrastructure	Minimal	& Minimal	Élevé
Complexité opérationnelle	Faible	Modérée	Forte

& Approche retenue : Schema-per-tenant (t_boutique_koua). Les tables métier sont dans le schema middleware fixe SET search_path =

Pourquoi pas tenant_id partout ?

- 30+ ALTER TABLE + 30+ politiques
- Un oubli de WHERE tenant_id=
- Les référentiels (types_clients,
- Impossible de faire un pg_dump

2. Fichiers de migration à créer

Vue d'ensemble

Fichier	Appliqué	Contenu
100_saas_tenants.sql	1 fois (serveur)	Tables tenants, plans, invitations
101_saas_provision.sql	1 fois (serveur)	Fonction provision_tenant()
102_tenant_schema_template.sql	Par nouveau client	Toutes les tables métier dans t_{slug}
103_saas_super_admin.sql	1 fois (serveur)	Compte super-admin global

Migration 100 — Tables globales (schéma public)

```
-- 100_saas_tenants.sql

CREATE TABLE public.tenants (
  id          SERIAL PRIMARY KEY,
  slug        VARCHAR(50) UNIQUE NOT NULL, -- ex: acme !' acme.telepro.ci
  nom         VARCHAR(255) NOT NULL,
  email_admin VARCHAR(255) NOT NULL,
  plan        VARCHAR(20) NOT NULL DEFAULT 'starter'
             CHECK (plan IN ('starter','pro','enterprise')),
  statut      VARCHAR(20) NOT NULL DEFAULT 'trial'
             CHECK (statut IN ('trial','actif','suspendu','resilié')),
  trial_ends_at TIMESTAMPTZ DEFAULT NOW() + INTERVAL '14 days',
  schema_name VARCHAR(63) GENERATED ALWAYS AS ('t_' || slug) STORED,
  created_at   TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE public.plans (
  code          VARCHAR(20) PRIMARY KEY,
  label         VARCHAR(100),
  max_magasins  INT DEFAULT 1,
  max_utilisateurs INT DEFAULT 5,
  max_produits  INT DEFAULT 500,
  prix_mensuel  NUMERIC(10,0), -- FCFA
  fonctionnalites JSONB DEFAULT '{}'
);

INSERT INTO public.plans VALUES
('starter', 'Starter', 1, 5, 500, 15000, '{"sms":false}'),
('pro', 'Pro', 3, 20, 5000, 45000, '{"sms":true,"rapports":true}'),
('enterprise', 'Enterprise', 99, 999, 99999, null, '{"sms":true,"api":true}');

CREATE TABLE public.invitations (
  id          SERIAL PRIMARY KEY,
  tenant_id   INT NOT NULL REFERENCES public.tenants(id) ON DELETE CASCADE,
  email       VARCHAR(255) NOT NULL,
  token       VARCHAR(100) UNIQUE NOT NULL
             DEFAULT encode(gen_random_bytes(32), 'hex'),
  role_code   VARCHAR(50) NOT NULL DEFAULT 'commercial',
  expires_at  TIMESTAMPTZ DEFAULT NOW() + INTERVAL '72 hours',
  used_at     TIMESTAMPTZ,
  created_at  TIMESTAMPTZ DEFAULT NOW()
);
```

2. Fichiers de migration (suite)

Migration 101 — Fonction de provisioning

```
-- 101_saas_provision.sql

CREATE OR REPLACE FUNCTION public.provision_tenant(
    p_slug          TEXT,
    p_nom           TEXT,
    p_email_admin   TEXT,
    p_plan          TEXT DEFAULT 'starter'
) RETURNS INT AS $$
DECLARE
    v_tenant_id INT;
    v_schema    TEXT;
BEGIN
    -- 1. Valider le slug (alphanumérique + tirets)
    IF p_slug !~ '^[a-z0-9][a-z0-9-]{1,48}[a-z0-9]$' THEN
        RAISE EXCEPTION 'Slug invalide : %', p_slug;
    END IF;

    -- 2. Créer la ligne tenant
    INSERT INTO public.tenants (slug, nom, email_admin, plan)
    VALUES (p_slug, p_nom, p_email_admin, p_plan)
    RETURNING id INTO v_tenant_id;

    v_schema := 't_' || p_slug;

    -- 3. Créer le schéma PostgreSQL isolé
    EXECUTE format('CREATE SCHEMA %I', v_schema);

    -- 4. Le runner Node.js applique 102_tenant_schema_template.sql
    -- avec SET search_path = {schema}, public

    RETURN v_tenant_id;
END;
$$ LANGUAGE plpgsql;

-- Fonction inverse : supprimer un tenant et toutes ses données
CREATE OR REPLACE FUNCTION public.deprovision_tenant(p_slug TEXT)
RETURNS VOID AS $$
DECLARE v_schema TEXT := 't_' || p_slug;
BEGIN
    EXECUTE format('DROP SCHEMA %I CASCADE', v_schema);
    DELETE FROM public.tenants WHERE slug = p_slug;
END;
$$ LANGUAGE plpgsql;
```

Migration 102 — Template schéma tenant (extrait)

```

-- 102_tenant_schema_template.sql
-- Exécuté avec : SET search_path = t_{slug}, public

-- Toutes les tables de 001!016 sans les données seed de demo
CREATE TABLE magasins ( id SERIAL PRIMARY KEY, code VARCHAR(20) UNIQUE, ... );
CREATE TABLE roles     ( id SERIAL PRIMARY KEY, code VARCHAR(50), permissions JSONB, ... );
CREATE TABLE utilisateurs ( id SERIAL PRIMARY KEY, email VARCHAR(255) UNIQUE, ... );
CREATE TABLE sessions ( id SERIAL PRIMARY KEY, user_id INT REFERENCES utilisateurs(id), ... );
CREATE TABLE produits ( ... );
CREATE TABLE stocks    ( produit_id INT, magasin_id INT, PRIMARY KEY(produit_id, magasin_id), ... );
-- ... toutes les tables existantes ...

-- Données initiales par défaut (pas de données de démo)
INSERT INTO roles (code, libelle, permissions) VALUES
  ('admin',      'Administrateur', '{"all":true}'),
  ('commercial', 'Commercial',     '{"ventes":true,"clients":true}'),
  ('caissier',   'Caissier',       '{"paiements":true}'),
  ('comptable',  'Comptable',      '{"rapports":true}'),
  ('magasinier', 'Magasinier',     '{"stock":true}');

INSERT INTO magasins (code, nom, actif) VALUES ('MG-001', 'Magasin Principal', TRUE);

INSERT INTO moyens_paiement (code, libelle, actif) VALUES
  ('ESPECES',      'Espèces',      TRUE),
  ('WAVE',         'Wave',         TRUE),
  ('MTNMOMO',      'MTN MoMo',     TRUE),
  ('ORANGEMONEY',  'Orange Money', TRUE),
  ('VIREMENT',     'Virement',     TRUE);

INSERT INTO societe_parametres (nom, email, pays, devise, tva_default)
VALUES (:tenant_nom, :tenant_email, 'Côte d'Ivoire', 'XOF', 18.00);

-- Triggers identiques à 008_paiements_trigger.sql
CREATE TRIGGER trg_paiements_sync_ventes AFTER INSERT OR UPDATE OR DELETE ON paiements ...

```

3. Modifications côté API

Middleware tenant.ts — résolution par sous-domaine

```
// src/middleware/tenant.ts

export async function resolveTenant(req, res, next) {
  // acme.telepro.ci  !' slug = "acme"
  const slug = req.hostname.split('.')[0];
  const schema = `t_${slug}`;

  const { rows } = await pool.query(
    `SELECT id, statut, plan FROM public.tenants WHERE slug=$1`, [slug]
  );
  if (!rows.length) return fail(res, 'Entreprise inconnue', 404);
  if (rows[0].statut==='suspendu') return fail(res, 'Compte suspendu', 402);
  if (rows[0].statut==='résilié') return fail(res, 'Abonnement résilié', 402);

  // Fixer le search_path pour toute la durée de la requête
  await pool.query(`SET LOCAL search_path = ${schema}, public`);

  req.tenant = { id: rows[0].id, slug, schema, plan: rows[0].plan };
  next();
}
```

Position dans la chaîne Express

```
// src/index.ts

app.use(cors());
app.use(express.json());
app.use(resolveTenant); // !• insérer ICI, avant toutes les routes
app.use('/api/auth', authRouter);
app.use('/api', requireAuth, mainRouter);
```

JWT étendu avec tenant_id

```
// src/routes/auth.ts – login

const payload = {
  sub,
  email, nom, prenom,
  role: user.role_code,
  perms: user.permissions,
  magasin_ids: user.magasin_ids,
  tenant_id: req.tenant.id, // !• nouveau
  tenant_slug: req.tenant.slug, // !• nouveau
};
```

Nouvelles routes à créer

Méthode + Route	Auth	Description
POST /api/tenants/register	Public	Inscription client !" provision_tenant()
GET /api/tenants/me	JWT tenant	Infos abonnement du client courant
PUT /api/tenants/me	Admin tenant	Modifier nom / email société
POST /api/tenants/invite	Admin tenant	Envoyer une invitation par email
POST /api/auth/accept-invite	Token email	Créer son compte via invitation
GET /api/super/tenants	Super-admin	Liste tous les tenants
PUT /api/super/tenants/:id	Super-admin	Changer plan, suspendre
DELETE /api/super/tenants/:id	Super-admin	deprovision_tenant() + suppression

Runner de provisioning Node.js

```
// src/services/provisionTenant.ts

export async function provisionTenant({ slug, nom, email, plan, passwordHash }) {
  const client = await pool.connect();
  try {
    await client.query('BEGIN');

    // 1. Appeler la fonction SQL (crée le schéma)
    const { rows } = await client.query(
      `SELECT public.provision_tenant($1,$2,$3,$4) AS tenant_id`,
      [slug,nom, email, plan]
    );
    const tenantId = rows[0].tenant_id;

    // 2. SET search_path sur le nouveau schéma
    await client.query(`SET search_path = t_${slug}, public`);

    // 3. Appliquer le template de schéma
    const sql = fs.readFileSync('migrations/102_tenant_schema_template.sql', 'utf8');
    await client.query(sql);

    // 4. Créer le compte admin du tenant
    await client.query(
      `INSERT INTO utilisateurs (code, nom, prenom, email, password_hash, role_id, actif)
      VALUES ('ADM-001', $1, '', $2, $3,
              (SELECT id FROM roles WHERE code='admin'), TRUE)`,
      [nom, email, passwordHash]
    );

    await client.query('COMMIT');
    return tenantId;
  } catch (e) {
    await client.query('ROLLBACK');
    throw e;
  } finally {
    client.release();
  }
}
```

4. Infrastructure & déploiement

Nginx — routing wildcard sous-domaine

```
# /etc/nginx/sites-enabled/telepro-saas.conf

server {
    listen 443 ssl;
    server_name *.telepro.ci;

    ssl_certificate      /etc/letsencrypt/live/telepro.ci/fullchain.pem;
    ssl_certificate_key  /etc/letsencrypt/live/telepro.ci/privkey.pem;

    location /api/ {
        proxy_pass        http://localhost:3001;
        proxy_set_header  Host $host;           # transmet acme.telepro.ci
        proxy_set_header  X-Real-IP $remote_addr;
    }

    location / {
        proxy_pass        http://localhost:3000;
        proxy_set_header  Host $host;
    }
}

# Certificat wildcard Let's Encrypt
# certbot certonly --dns-infomaniak -d telepro.ci -d *.telepro.ci
```

Tableau des composants infrastructure

Composant	Outil	Rôle
Reverse proxy	Nginx	Routing *.telepro.ci !' API/Web
TLS wildcard	Let's Encrypt + DNS plugin	Certificat *.telepro.ci automatique
Pool connexions	pg-pool (existant)	search_path par requête (SET LOCAL)
Email transac.	Resend ou Mailgun	Invitations, bienvenue, factures
Billing	Stripe ou paiement mobile	Abonnements, renouvellements FCFA
Backup tenants	pg_dump -n t_{slug} + cron	Isolation RGPD par client
Monitoring	Grafana + Loki	Métriques par tenant (slug en label)
CI/CD migration	Script Node provisionTenant	Onboarding automatisé sans downtime

Docker Compose (ajouts)


```
# docker-compose.yml – services supplémentaires

services:
  postgres:
    image: postgres:16
    environment:
      POSTGRES_DB: telepro_saas    # base unique, schémas multiples

  api:
    environment:
      DATABASE_URL: postgresql://user:pass@postgres:5432/telepro_saas
      SUPER_ADMIN_KEY: ${SUPER_ADMIN_KEY}    # clé pour les routes /super/*

# Nouveau : service d'onboarding (optionnel, peut être dans l'API)
onboarding:
  build: ./apps/onboarding
  ports: ['3002:3002']
  environment:
    DATABASE_URL: ${DATABASE_URL}
    MAIL_FROM: noreply@telepro.ci
```

5. Plans d'abonnement et quotas

Tableau des plans

	Starter	Pro	Enterprise
Prix/mois (FCFA)	15 000	45 000	Sur devis
Magasins	1	3	Illimité
Utilisateurs	5	20	Illimité
Produits catalogue	500	5 000	Illimité
SMS / WhatsApp	'	'	'
Rapports avancés	'	'	'
Export PDF	'	'	'
API REST accès direct	'	'	'
SLA support	Email	24h	4h
Backup quotidien	'	'	' + restore

Middleware de vérification des quotas

```
// src/middleware/quotas.ts

export function checkQuota(ressource: string) {
  return async (req, res, next) => {
    const plan = await getPlanLimits(req.tenant.plan);

    if (ressource === 'magasins') {
      const { rows } = await pool.query(`SELECT COUNT(*) n FROM magasins WHERE actif`);
      if (Number(rows[0].n) >= plan.max_magasins)
        return fail(res, `Limite atteinte (${plan.max_magasins} magasins sur plan ${req.tenant.plan})`, 402);

      if (ressource === 'utilisateurs') {
        const { rows } = await pool.query(`SELECT COUNT(*) n FROM utilisateurs WHERE actif`);
        if (Number(rows[0].n) >= plan.max_utilisateurs)
          return fail(res, `Limite utilisateurs atteinte`, 402);
      }

      next();
    }
  };
}

// Usage dans les routes :
router.post('/magasins', requireAdmin, checkQuota('magasins'), wrap(creerMagasin));
router.post('/admin/utilisateurs', requireAdmin, checkQuota('utilisateurs'), wrap(creerUtilisateur));
```

6. Plan de migration progressif

La migration peut se faire sans interruption de service pour le client existant. Le client existant (tenant) devient le premier tenant du SaaS.

Phase 1 — Préparation (semaine 1)

- Créer les migrations 100 et 101 (tables globales + fonction provision)
- Écrire le template 102 à partir des migrations existantes 001!016
- Mettre en place le runner provisionTenant() dans les services API
- Configurer le certificat wildcard Let's Encrypt (*.telepro.ci)
- Tester en local avec deux sous-domaines : demo.localhost et client1.localhost

Phase 2 — Migration du client existant (semaine 2)

- Provisionner le schéma t_telepro pour le client actuel
- Migrer les données existantes : INSERT INTO t_telepro.ventes SELECT * FROM t_telepro.ventes
- Vérifier les vues, triggers et séquences dans le nouveau schéma
- Basculer le sous-domaine principal : telepro.telepro.ci (ou main.telepro.ci)
- Conserver public.* en read-only pendant 2 semaines (rollback possible)

Phase 3 — Ouverture SaaS (semaine 3-4)

- Activer la route POST /api/tenants/register avec le formulaire d'inscription
- Mettre en place l'envoi d'emails de bienvenue (Resend / Mailgun)
- Intégrer la page de pricing sur le site marketing
- Tester le flow complet : inscription !' onboarding !' premier login
- Mettre en place les alertes de monitoring (Grafana par tenant)

Phase 4 — Billing (semaine 5-6)

- Intégrer Stripe ou paiement Wave/MTN MoMo pour les abonnements
- Automatiser la suspension après échec de paiement (statut = suspendu)
- Mettre en place les rappels email J-7, J-3, J-0 avant expiration
- Activer les quotas par plan (checkQuota middleware)

Tableau de suivi de migration

Tâche	Complexité	Impact code existant	Semaine
Tables globales (100, 101)	Faible	Aucun	S1
Template schéma (102)	Moyenne	Aucun	S1
Middleware resolveTenant	Faible	src/index.ts	S1
JWT étendu tenant_id	Faible	src/routes/auth	S2
Cert wildcard + Nginx	Faible	Infra seule	S1
Migration données existantes	Moyenne	Script one-shot	S2
Routes /tenants/* + /super/*	Moyenne	Nouvelles routes	S3
Quotas par plan	Faible	+1 middleware	S3
Billing Stripe/mobile	Forte	Nouveau service	S5

7. Sécurité et conformité RGPD

Isolation garantie par PostgreSQL

L'isolation `search_path` est une garantie au niveau du moteur de base de données : dans le contexte `t_acme` ne peut physiquement pas lire les tables du schéma `t_a` explicite. Le super-user PostgreSQL conserve un accès global pour la maintenance.

- `SET LOCAL search_path` : l'isolation est annulée automatiquement en fin de requête
- Aucune FK cross-schéma : impossible de joindre accidentellement deux tenants
- Séquences indépendantes : les IDs recommencent à 1 par tenant (pas de fuite)

Droit à l'effacement (Art. 17 RGPD)

```
-- Suppression complète d'un tenant et de toutes ses données
SELECT public.deprovision_tenant('acme');
-- !' DROP SCHEMA t_acme CASCADE (toutes les tables + données)
-- !' DELETE FROM public.tenants WHERE slug='acme'
-- !' DELETE FROM public.invitations WHERE tenant_id=...
-- Durée estimée : < 5 secondes pour 500 000 lignes
```

Portabilité des données (Art. 20 RGPD)

```
# Export des données d'un tenant en SQL
pg_dump -n t_acme -Fc telepro_saas > acme_backup_2026.dump

# Restauration chez un autre hébergeur
pg_restore -n t_acme -d nouvelle_base acme_backup_2026.dump
```

Sécurité supplémentaire recommandée

Chiffrement at-rest PostgreSQL `pgcrypto`
pour colonnes sensibles (RCCM, IBAN)

Audit cross-tenant Table
`public.audit_global` (slug, action, ip, timestamp)

Rate limiting `nginx limit_req_zone`
`$http_host` (par sous-domaine)

Monitoring anomalies Alerte si un token
accède à un `schema_name` "tenant JWT"

Rotation JWT secrets Secret JWT par
tenant (permet révocation ciblée)

8. Bilan et récapitulatif

Ce qui change dans le code actuel

```
src/  
ind  
ex.t  
s :  
ajo  
ut  
de  
app  
.us  
e(re  
solv  
eTe  
nan  
t)  
src/  
rout  
es/  
aut  
h.ts  
: te  
nan  
t_id  
da  
ns  
le J  
WT  
src/  
mid  
dle  
war  
e/a  
uth.  
ts :  
véri  
fica  
tion  
ten  
ant  
_id  
dan  
s re  
quir  
eAu  
th  
'To  
ute  
s  
les  
rout  
es  
mét  
ier (  
ven  
tes,  
sto  
ck  
...)
```

: A
UC
UN
cha
nge
me
nt
'Mi
grat
ion
s 0
01!'0
16 :
AU
CU
N c
han
ge
me
nt (r
éuti
lisé
es
dan
s
le t
em
plat
e)
'Fro
nte
nd
Re
act
: aj
out
du
sou
s-d
om
ain
e d
ans
axi
os
bas
eU
RL

Ce qui est entièrement nouveau

- 3 fichiers SQL de migration (100, 101, 102)
- Middleware resolveTenant (environ 25 lignes)
- Service provisionTenant (environ 50 lignes)
- Routes /api/tenants/* et /api/super/* (environ 100 lignes)
- Middleware checkQuota (environ 40 lignes)
- Configuration Nginx wildcard + certificat Let's Encrypt

Estimation effort

Tâche	Jours développeur
Migrations SQL (100, 101, 102)	2 j
Middleware + routes API	3 j
Runner provisionTenant + tests	2 j
Infra Nginx + TLS wildcard	1 j
Migration données client existant	1 j
Billing (Stripe ou Wave/MTN)	4 j
Tests end-to-end onboarding	2 j
Total estimé (hors billing)	9 j
Total estimé (avec billing)	13 j

L'architecture schema-per-tenant est la
ne demande pas de modifier le code m
pour tous les tenants). Le chemin critiq
— le reste est incrémental et peut être